

CS3014 Estructuras de Datos Avanzadas

Laboratorio - Semana 9 - Sesión 2

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

22 de mayo de 2026

¿Qué aprenderemos hoy?



¿Qué aprenderemos hoy?



- Equivalencia entre ordenamiento y colas de prioridad

¿Qué aprenderemos hoy?



- ▶ Equivalencia entre ordenamiento y colas de prioridad
- ▶ Algunos resultados en ordenamiento de enteros

¿Qué aprenderemos hoy?



- ▶ Equivalencia entre ordenamiento y colas de prioridad
- ▶ Algunos resultados en ordenamiento de enteros
- ▶ Signature sort

Ordenamiento y cola de prioridades

Equivalencia



Punto inicial

Equivalencia



Punto inicial

Es sencillo notar que si tenemos una cola de prioridad que soporta las operaciones clásicas en $O(f(n, w))$, entonces existe un algoritmo de ordenamiento que toma $O(nf(n, w))$.

Equivalencia



Punto inicial

Es sencillo notar que si tenemos una cola de prioridad que soporta las operaciones clásicas en $O(f(n, w))$, entonces existe un algoritmo de ordenamiento que toma $O(nf(n, w))$.

Punto final

Equivalencia



Punto inicial

Es sencillo notar que si tenemos una cola de prioridad que soporta las operaciones clásicas en $O(f(n, w))$, entonces existe un algoritmo de ordenamiento que toma $O(nf(n, w))$.

Punto final

Si existe un algoritmo de ordenamiento que toma $O(T(n, w))$, existe una cola de prioridad que soporta sus operaciones clásicas en $O\left(\frac{T(n, w)}{n}\right)$.

Equivalencia



Punto inicial

Es sencillo notar que si tenemos una cola de prioridad que soporta las operaciones clásicas en $O(f(n, w))$, entonces existe un algoritmo de ordenamiento que toma $O(nf(n, w))$.

Punto final

Si existe un algoritmo de ordenamiento que toma $O(T(n, w))$, existe una cola de prioridad que soporta sus operaciones clásicas en $O\left(\frac{T(n, w)}{n}\right)$.

Ejemplo: Quickheap.

Resultados en ordenamientos de enteros

Resultados en ordenamiento de enteros



Resultados en ordenamiento de enteros



Para ordenar n enteros de w bits, la siguiente tabla resume algunos resultados históricos.

Resultados en ordenamiento de enteros

Para ordenar n enteros de w bits, la siguiente tabla resume algunos resultados históricos.

Método	Complejidad	Condición
Comparaciones	$O(n \log n)$	Sin condiciones
Counting sort	$O(n + 2^w)$	$O(n)$ si $w = \log n$
Radix sort	$O\left(n \frac{w}{\log n}\right)$	$O(n)$ si $w = O(\log n)$
van Emde Boas	$O(n \log w)$	
Basado en van Emde Boas	$O(n \log \frac{w}{\log n})$	$O(n \log \log n)$ si $w = \log^{O(1)} n$
Signature sort	$O(n)$	$w = \Omega(\log^{2+\varepsilon} n), \forall \varepsilon > 0$
Han (2004)	$O(n \log \log n)$	modelo RAM AC^0
Han y Thorup (2002)	$O(n \sqrt{\log \frac{w}{\log n}})$	Aleatorizado

Cuadro: Resultados de ordenamiento de enteros

Signature sort

Signature sort



Condición base

Signature sort



Condición base

Asumiremos que $w \geq \log^{2+\epsilon} n \log \log n$ por ser conveniente.

Signature sort



Condición base

Asumiremos que $w \geq \log^{2+\epsilon} n \log \log n$ por ser conveniente.

Paso 1

Signature sort



Condición base

Asumiremos que $w \geq \log^{2+\epsilon} n \log \log n$ por ser conveniente.

Paso 1

Particionamos cada entero en $\log^\epsilon n$ bloques de tamaños iguales que serán dígitos.

Signature sort



Condición base

Asumiremos que $w \geq \log^{2+\epsilon} n \log \log n$ por ser conveniente.

Paso 1

Particionamos cada entero en $\log^\epsilon n$ bloques de tamaños iguales que serán dígitos.

Notemos que estos bloques tendrán tamaño de al menos $\log^2 n$.

Signature sort



Condición base

Asumiremos que $w \geq \log^{2+\epsilon} n \log \log n$ por ser conveniente.

Paso 1

Particionamos cada entero en $\log^\epsilon n$ bloques de tamaños iguales que serán dígitos.

Notemos que estos bloques tendrán tamaño de al menos $\log^2 n$.

Esto es equivalente a tomar una base $B \approx \frac{w}{\log^\epsilon n}$ y representar cada entero en base B .

Signature sort



Paso 2

Signature sort



Paso 2

Ya que hay pocos posibles dígitos almacenados para un universo muy grande ($n \log^{1+\varepsilon} n \ll 2^{\log^2 n}$), reemplazamos cada bloque por un hash perfecto de $O(\log n)$ bits. Notemos que podríamos elegir una constante grande para evitar colisiones con alta probabilidad.

Signature sort



Paso 2

Ya que hay pocos posibles dígitos almacenados para un universo muy grande ($n \log^{1+\varepsilon} n \ll 2^{\log^2 n}$), reemplazamos cada bloque por un hash perfecto de $O(\log n)$ bits. Notemos que podríamos elegir una constante grande para evitar colisiones con alta probabilidad.

Podemos multiplicar cada dígito por un entero m y luego obtener las máscaras reducidas de cada uno. Esto tomará $O(1)$ por dígito.

Signature sort



Paso 2

Ya que hay pocos posibles dígitos almacenados para un universo muy grande ($n \log^{1+\varepsilon} n \ll 2^{\log^2 n}$), reemplazamos cada bloque por un hash perfecto de $O(\log n)$ bits. Notemos que podríamos elegir una constante grande para evitar colisiones con alta probabilidad.

Podemos multiplicar cada dígito por un entero m y luego obtener las máscaras reducidas de cada uno. Esto tomará $O(1)$ por dígito. Esto preserva la identidad pero no preserva el orden.

Signature sort



Paso 2

Ya que hay pocos posibles dígitos almacenados para un universo muy grande ($n \log^{1+\varepsilon} n \ll 2^{\log^2 n}$), reemplazamos cada bloque por un hash perfecto de $O(\log n)$ bits. Notemos que podríamos elegir una constante grande para evitar colisiones con alta probabilidad.

Podemos multiplicar cada dígito por un entero m y luego obtener las máscaras reducidas de cada uno. Esto tomará $O(1)$ por dígito. Esto preserva la identidad pero no preserva el orden.

Paso 3

Signature sort



Paso 2

Ya que hay pocos posibles dígitos almacenados para un universo muy grande ($n \log^{1+\epsilon} n \ll 2^{\log^2 n}$), reemplazamos cada bloque por un hash perfecto de $O(\log n)$ bits. Notemos que podríamos elegir una constante grande para evitar colisiones con alta probabilidad.

Podemos multiplicar cada dígito por un entero m y luego obtener las máscaras reducidas de cada uno. Esto tomará $O(1)$ por dígito. Esto preserva la identidad pero no preserva el orden.

Paso 3

Notemos que ahora la cantidad de bits usados es $O(n \log^{1+\epsilon} n)$, debido a que tenemos n enteros con $\log^\epsilon n$ dígitos de $\log n$ bits.

Signature sort



Packed sort

Signature sort



Packed sort

Es un algoritmo que nos permite ordenar n enteros de b bits en $O(n)$ si el tamaño de la palabra cumple $w = \Omega(b \log n \log \log n)$.

Signature sort



Packed sort

Es un algoritmo que nos permite ordenar n enteros de b bits en $O(n)$ si el tamaño de la palabra cumple $w = \Omega(b \log n \log \log n)$.

El paso 2 nos dejó con enteros de $O(\log^{1+\epsilon} n)$ bits, así que cumplimos la condición sin problemas sobre los hashes/signatures. Por lo tanto, ordenaremos los hashes en $O(n)$.

Signature sort



Packed sort

Es un algoritmo que nos permite ordenar n enteros de b bits en $O(n)$ si el tamaño de la palabra cumple $w = \Omega(b \log n \log \log n)$.

El paso 2 nos dejó con enteros de $O(\log^{1+\epsilon} n)$ bits, así que cumplimos la condición sin problemas sobre los hashes/signatures. Por lo tanto, ordenaremos los hashes en $O(n)$.

Paso 4

Signature sort



Packed sort

Es un algoritmo que nos permite ordenar n enteros de b bits en $O(n)$ si el tamaño de la palabra cumple $w = \Omega(b \log n \log \log n)$.

El paso 2 nos dejó con enteros de $O(\log^{1+\epsilon} n)$ bits, así que cumplimos la condición sin problemas sobre los hashes/signatures. Por lo tanto, ordenaremos los hashes en $O(n)$.

Paso 4

Construimos un trie comprimido sobre los hashes, de manera que el inorder traversal del mismo nos da los hashes ordenados.

Signature sort



Packed sort

Es un algoritmo que nos permite ordenar n enteros de b bits en $O(n)$ si el tamaño de la palabra cumple $w = \Omega(b \log n \log \log n)$.

El paso 2 nos dejó con enteros de $O(\log^{1+\epsilon} n)$ bits, así que cumplimos la condición sin problemas sobre los hashes/signatures. Por lo tanto, ordenaremos los hashes en $O(n)$.

Paso 4

Construimos un trie comprimido sobre los hashes, de manera que el inorder traversal del mismo nos da los hashes ordenados.

Esto toma $O(n)$ de espacio y es posible construirlo en $O(n)$ también.

Signature sort



Trie comprimido

Signature sort



Trie comprimido

Tomaremos cada nodo del trie con alfabeto $\{0, 1, \dots, B - 1\}$ que no tiene al menos dos hijos almacenados y lo “uniremos” con su padre.

Signature sort



Trie comprimido

Tomaremos cada nodo del trie con alfabeto $\{0, 1, \dots, B - 1\}$ que no tiene al menos dos hijos almacenados y lo “uniremos” con su padre.

Debido a lo anterior, tendremos $O(n)$ nodos en la versión comprimida, comparado con los $O(n \log^\epsilon n)$ que tendríamos en la versión completa.

Signature sort



Trie comprimido

Tomaremos cada nodo del trie con alfabeto $\{0, 1, \dots, B-1\}$ que no tiene al menos dos hijos almacenados y lo “uniremos” con su padre.

Debido a lo anterior, tendremos $O(n)$ nodos en la versión comprimida, comparado con los $O(n \log^\epsilon n)$ que tendríamos en la versión completa.

Problemática

Signature sort



Trie comprimido

Tomaremos cada nodo del trie con alfabeto $\{0, 1, \dots, B - 1\}$ que no tiene al menos dos hijos almacenados y lo “uniremos” con su padre.

Debido a lo anterior, tendremos $O(n)$ nodos en la versión comprimida, comparado con los $O(n \log^\epsilon n)$ que tendríamos en la versión completa.

Problemática

Tenemos todos los enteros originales ordenados según sus hashes, así que en realidad no tenemos ningún orden correcto todavía.

Signature sort



Paso 5

Signature sort



Paso 5

Ordenaremos las aristas del trie recursivamente usando (Índice del nodo, Primer dígito, Índice del hijo).

Signature sort



Paso 5

Ordenaremos las aristas del trie recursivamente usando (Índice del nodo, Primer dígito, Índice del hijo).

De esta manera, si el nivel actual tiene enteros de w' bits, estamos reduciéndolos a una tupla de $(O(\log n), O(\frac{w'}{\log^\epsilon n}), O(\log n))$, así que la cantidad de bits se reduce a $\frac{w'}{\log^\epsilon n} + O(\log n)$

Signature sort



Paso 5

Ordenaremos las aristas del trie recursivamente usando (Índice del nodo, Primer dígito, Índice del hijo).

De esta manera, si el nivel actual tiene enteros de w' bits, estamos reduciéndolos a una tupla de $(O(\log n), O(\frac{w'}{\log^\epsilon n}), O(\log n))$, así que la cantidad de bits se reduce a $\frac{w'}{\log^\epsilon n} + O(\log n)$

Luego de $\frac{1}{\epsilon} + 1$ niveles de recursión se tendrá que la cantidad de bits de cada entero es $b = O(\frac{1}{\epsilon} \log n + \frac{w'}{\log^{1+\epsilon} n}) = O(\frac{w}{\log n \log \log n})$.

Signature sort



Paso 5

Ordenaremos las aristas del trie recursivamente usando (Índice del nodo, Primer dígito, Índice del hijo).

De esta manera, si el nivel actual tiene enteros de w' bits, estamos reduciéndolos a una tupla de $(O(\log n), O(\frac{w'}{\log^\epsilon n}), O(\log n))$, así que la cantidad de bits se reduce a $\frac{w'}{\log^\epsilon n} + O(\log n)$

Luego de $\frac{1}{\epsilon} + 1$ niveles de recursión se tendrá que la cantidad de bits de cada entero es $b = O(\frac{1}{\epsilon} \log n + \frac{w'}{\log^{1+\epsilon} n}) = O(\frac{w}{\log n \log \log n})$.

Sobre este escenario podemos usar packed sort y ordenar todo en $O(n)$ como caso base.

Signature sort



Paso 5

Ordenaremos las aristas del trie recursivamente usando (Índice del nodo, Primer dígito, Índice del hijo).

De esta manera, si el nivel actual tiene enteros de w' bits, estamos reduciéndolos a una tupla de $(O(\log n), O(\frac{w'}{\log^\epsilon n}), O(\log n))$, así que la cantidad de bits se reduce a $\frac{w'}{\log^\epsilon n} + O(\log n)$

Luego de $\frac{1}{\epsilon} + 1$ niveles de recursión se tendrá que la cantidad de bits de cada entero es $b = O(\frac{1}{\epsilon} \log n + \frac{w'}{\log^{1+\epsilon} n}) = O(\frac{w}{\log n \log \log n})$.

Sobre este escenario podemos usar packed sort y ordenar todo en $O(n)$ como caso base.

Paso 6

Signature sort



Paso 5

Ordenaremos las aristas del trie recursivamente usando (Índice del nodo, Primer dígito, Índice del hijo).

De esta manera, si el nivel actual tiene enteros de w' bits, estamos reduciéndolos a una tupla de $(O(\log n), O(\frac{w'}{\log^\epsilon n}), O(\log n))$, así que la cantidad de bits se reduce a $\frac{w'}{\log^\epsilon n} + O(\log n)$

Luego de $\frac{1}{\epsilon} + 1$ niveles de recursión se tendrá que la cantidad de bits de cada entero es $b = O(\frac{1}{\epsilon} \log n + \frac{w'}{\log^{1+\epsilon} n}) = O(\frac{w}{\log n \log \log n})$.

Sobre este escenario podemos usar packed sort y ordenar todo en $O(n)$ como caso base.

Paso 6

Permutaremos los hijos de cada nodo según el orden de sus aristas.

Signature sort



Paso 7

Signature sort



Paso 7

Ya que ahora tendremos los hijos de cada nodo del trie comprimido correctamente ordenados según el valor original de sus datos, podemos ordenar calculando el inorder traversal del trie.

Signature sort



Paso 7

Ya que ahora tendremos los hijos de cada nodo del trie comprimido correctamente ordenados según el valor original de sus datos, podemos ordenar calculando el inorder traversal del trie.

En resumen

Signature sort



Paso 7

Ya que ahora tendremos los hijos de cada nodo del trie comprimido correctamente ordenados según el valor original de sus datos, podemos ordenar calculando el inorder traversal del trie.

En resumen

1. Dividimos cada entero en dígitos para poder tener valores más pequeños.

Signature sort



Paso 7

Ya que ahora tendremos los hijos de cada nodo del trie comprimido correctamente ordenados según el valor original de sus datos, podemos ordenar calculando el inorder traversal del trie.

En resumen

1. Dividimos cada entero en dígitos para poder tener valores más pequeños.
2. Aplicamos hashing para poder reducir el espacio usado para representar los dígitos.

Signature sort



Paso 7

Ya que ahora tendremos los hijos de cada nodo del trie comprimido correctamente ordenados según el valor original de sus datos, podemos ordenar calculando el inorder traversal del trie.

En resumen

1. Dividimos cada entero en dígitos para poder tener valores más pequeños.
2. Aplicamos hashing para poder reducir el espacio usado para representar los dígitos.
3. Usamos un trie comprimido para “ordenar” los hashes

Signature sort



Paso 7

Ya que ahora tendremos los hijos de cada nodo del trie comprimido correctamente ordenados según el valor original de sus datos, podemos ordenar calculando el inorder traversal del trie.

En resumen

1. Dividimos cada entero en dígitos para poder tener valores más pequeños.
2. Aplicamos hashing para poder reducir el espacio usado para representar los dígitos.
3. Usamos un trie comprimido para “ordenar” los hashes
4. Ordenamos recursivamente las aristas del trie comprimido para ordenarlos según el valor original.

Signature sort



Paso 7

Ya que ahora tendremos los hijos de cada nodo del trie comprimido correctamente ordenados según el valor original de sus datos, podemos ordenar calculando el inorder traversal del trie.

En resumen

1. Dividimos cada entero en dígitos para poder tener valores más pequeños.
2. Aplicamos hashing para poder reducir el espacio usado para representar los dígitos.
3. Usamos un trie comprimido para “ordenar” los hashes
4. Ordenamos recursivamente las aristas del trie comprimido para ordenarlos según el valor original.
5. Acomodamos el trie según dicho orden y calculamos el inorder traversal.

Resumen de la sesión



Resumen de la sesión



- ▶ Aprendimos la equivalencia entre ordenamiento y colas de prioridad.

Resumen de la sesión



- ▶ Aprendimos la equivalencia entre ordenamiento y colas de prioridad.
- ▶ Aprendimos algunos resultados de ordenamientos de enteros.

Resumen de la sesión



- ▶ Aprendimos la equivalencia entre ordenamiento y colas de prioridad.
- ▶ Aprendimos algunos resultados de ordenamientos de enteros.
- ▶ Aprendimos signature sort.

Gracias